JAVASCRIPT ESTRUCTURAL Y DINÁMICO: DE ES6+ AL EVENT LOOP

Ingeniería Web Moderna: Sintaxis ES2025+, DOM Reactivo, Microtasks, Asincronía, Fetch API y Gestión de Estado

Manual técnico, metodológico y operativo para ingenieros de software frontend y full-stack. Elaborado bajo los estándares de desarrollo de software industrial de Onetel C.A. Este documento aborda la ingeniería moderna de JavaScript en profundidad: desde las estructuras de datos inmutables y la sintaxis ECMAScript contemporánea, pasando por el motor V8 y la mecánica del Event Loop (Call Stack, Web APIs, Microtasks vs. Macrotasks), hasta la manipulación reactiva del DOM con DocumentFragment, delegación de eventos, debouncing, consumo robusto de APIs REST con AbortController, y la implementación de una Single Page Application (SPA) ligera de producción.

Módulo	Marco Conceptual, Hitos Técnicos y Competencias	Estándares y Runtimes
Módulo 1	JavaScript Estructural: Tipos Primitivos, Inmutabilidad, Destructuring y Rest/Spread	ECMAScript 2025+ / V8 Engine
Módulo 2	Programación Funcional en JS: Funciones Puras, Métodos Inmutables (.map, .filter, .reduce)	Functional Programming
Módulo 3	Closures, Lexical Scope, Garbage Collection y Prevención de Fugas de Memoria	Memory Management
Módulo 4	Módulos Nativos (ESM): `import`, `export`, Dynamic Imports y Bundling	ES Modules Standard
Módulo 5	El DOM Dinámico: Manipulación Eficiente, DocumentFragment y Mitigación de Layout Thrashing	DOM Living Standard
Módulo 6	Arquitectura de Eventos: Fase de Captura, Burbujeo (Bubbling) y Event Delegation	Event Driven Architecture
Módulo 7	Patrones de Optimización de Entrada: Técnicas de Debouncing y Throttling	Performance Optimization
Módulo 8	El Event Loop al Descubierto: Call Stack, Web APIs, Microtask Queue vs Macrotasks	Concurrency Model
Módulo 9	Asincronía Moderna: Promesas, Async/Await, Error Boundaries y Cancelación (AbortController)	Async / Await Flow
Módulo 10	Fetch API Robusto: Backoff Exponencial, Manejo de Códigos HTTP y Detección XSS	Network Layer Security
Prácticas	Laboratorios Guiados: Buscador en Vivo, SPA de Telemetría y Checklist de Auditoría	Hands-On Code / 20 Criterios

Nombre del Desarrollador:

Empresa / Proyecto:

Cédula / ID Profesional:

Instructor Responsable:

1.1 TIPOS DE DATOS, INMUTABILIDAD Y SINTAXIS MODERNA ES2025+

JAVASCRIPT ESTRUCTURAL

Tipos Primitivos vs. Referencias en Memoria

Paso por Valor vs. Paso por Referencia: JavaScript maneja 7 tipos primitivos (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`) que se almacenan directamente en la pila (*Call Stack*) y se copian por valor. Los tipos complejos (`Object`, `Array`, `Function`) se almacenan en el montón (*Heap*) y las variables solo guardan un puntero de referencia en memoria.
El Peligro de la Mutación Accidental: Si asignas `const b = a` siendo a un objeto y modificas `b.status = 'OFFLINE'`, alterarás también a a. La ingeniería moderna exige tratar el estado como inmutable mediante operadores de dispersión (*spread*), desestructuración y clonado estructurado (`structuredClone()`).

JAVASCRIPT: DESESTRUCTURACIÓN, REST/SPREAD Y CLONADO PROFUNDO — estructural.js

```
// 1. Desestructuración de Objetos con Valores por Defecto y Renombrado
const servidor = {
  id: 101,
  hostname: "srv-fastapi-core",
  telemetria: { cpu: 14.2, ramMB: 1850 },
  ipLocal: "192.168.1.15"
};

// Extracción limpia sin acceder con punto repetitivo
const { hostname, telemetria: { cpu, ramMB: memoria = 0 }, estado = "ONLINE" } = servidor;

// 2. Operador Spread (...) para Inmutabilidad Superficial (Shallow Copy)
const servidorActualizado = {
  ...servidor,
  estado: "WARNING",
  puerto: 8000 // Nuevo campo sin mutar el objeto original 'servidor'
};

// 3. Clonado Estructurado Profundo (Deep Copy Seguro en ES2022+)
const copiaProfunda = structuredClone(servidor);
copiaProfunda.telemetria.cpu = 95.0; // NO afecta a servidor.telemetria.cpu

// 4. Parámetros Rest (...) para Funciones Variádicas
function registrarLogs(nivel, ...mensajes) {
  const timestamp = new Date().toISOString();
  mensajes.forEach(msg => console.log(`[${timestamp}] [${nivel.toUpperCase()}]: ${msg}`));
}
```

Comparación de Mutación vs Inmutabilidad

Operación	Enfoque Mutativo (Destructivo)	Enfoque Inmutable (Recomendado)
Añadir a un Array	`arr.push(item)` (Altera el array original)	`[...arr, item]` o `arr.concat(item)`
Remover de un Array	`arr.splice(index, 1)`	`arr.filter(item => item.id !== id)`
Modificar Propiedad	`obj.activo = false;`	`{ ...obj, activo: false }`
Ordenar Elementos	`arr.sort()` (Mutación en el lugar)	`arr.toSorted()` (ECMAScript 2023 inmutable)

2.1 PROGRAMACIÓN FUNCIONAL: FUNCIONES PURAS Y MÉTODOS DE ARRAY

FUNCTIONAL JS

Principios: Funciones Puras y Ausencia de Efectos Secundarios

¿Qué hace a una función pura?
1. **Determinismo:** Dado el mismo conjunto de entradas, siempre retornará el mismo resultado exacto.
2. **Cero Efectos Secundarios (Side Effects):** No modifica variables globales, no altera objetos pasados por referencia, no realiza I/O de red ni muta el DOM. Las funciones puras son triviales de testear, paralelizar y depurar.

JAVASCRIPT: PIPELINE DE TRANSFORMACIÓN CON MAP, FILTER, REDUCE Y TOSORTED — arrays-funcionales.js

```javascript
// Colección base inmutable de telemetría
const clusterServidores = Object.freeze([
  { id: 1, hostname: "srv-node-api", cpu: 12.5, ramMB: 1200, online: true },
  { id: 2, hostname: "srv-laravel-db", cpu: 84.0, ramMB: 7100, online: true },
  { id: 3, hostname: "srv-fastapi-worker", cpu: 0.0, ramMB: 0, online: false },
  { id: 4, hostname: "srv-nginx-proxy", cpu: 5.2, ramMB: 450, online: true }
]);

// 1. FILTER: Extraer únicamente servidores activos (Retorna un nuevo array)
const nodosActivos = clusterServidores.filter(s => s.online);

// 2. MAP: Transformar y normalizar a un nuevo esquema visual
const tarjetasDTO = nodosActivos.map(({ id, hostname, cpu, ramMB }) => ({
  nodoId: id,
  etiqueta: hostname.toUpperCase(),
  consumoCPU: `${cpu.toFixed(1)}%`,
  alerta: cpu > 80 ? "CRÍTICA" : "NORMAL"
}));

// 3. REDUCE: Reducir la colección a un único valor escalar (Promedio CPU)
const sumaCPU = nodosActivos.reduce((acumulador, nodo) => acumulador + nodo.cpu, 0);
const cpuPromedio = (sumaCPU / nodosActivos.length).toFixed(2);

// 4. FIND y SOME: Búsqueda rápida de elementos o existencia de condiciones
const nodoCritico = clusterServidores.find(s => s.cpu > 80);
const tieneNodosCaidos = clusterServidores.some(s => !s.online);

// 5. TOSORTED: Ordenamiento inmutable sin alterar el array de origen (ES2023)
const ordenadosPorCarga = clusterServidores.toSorted((a, b) => b.cpu - a.cpu);
```

Composición de Métodos (Method Chaining): Los métodos funcionales se pueden encadenar en una sola expresión declarativa legible: `cluster.filter(s => s.online).map(s => s.cpu).reduce((acc, v) => acc + v, 0)`, eliminando por completo bucles `for` manuales con variables contadoras propensas a errores de índice.

3.1 CLOSURES, ÁMBITO LÉXICO Y PREVENCIÓN DE FUGAS DE MEMORIA

CLOSURES & MEMORY OPS

Mecánica de los Closures y Encapsulamiento Privado

¿Qué es un Closure? Es la combinación de una función y el entorno léxico en el cual fue declarada. Un closure permite a una función interna **recordar y acceder** a las variables de su función contenedora padre, incluso mucho tiempo después de que la función padre haya finalizado su ejecución y salido del Call Stack. Constituye la base del encapsulamiento de estado privado antes de la introducción de los campos privados de clase (`#campo`).

JAVASCRIPT: FACTORY FUNCTION CON ESTADO PRIVADO Y CLEANUP — closures-factory.js

```javascript
// Fábrica de Gestores de Sesión con Estado Blindado
function crearMonitorServidor(idServidor) {
  // Variable privada en el Lexical Environment (inaccesible desde fuera)
  let contadorReintentos = 0;
  let intervaloId = null;

  return {
    registrarFallo() {
      contadorReintentos++;
      console.warn(`Nodo ${idServidor}: Fallo #${contadorReintentos}`);
      return contadorReintentos;
    },
    obtenerIntentos() {
      return contadorReintentos;
    },
    iniciarPolling(callback, intervaloMs = 3000) {
      if (intervaloId) return;
      intervaloId = setInterval(callback, intervaloMs);
    },
    // Protocolo explícito para evitar fugas de memoria (Memory Leaks)
    destruir() {
      if (intervaloId) {
        clearInterval(intervaloId);
        intervaloId = null;
        console.log(`Polling detenido para Nodo ${idServidor}. Memoria liberada.`);
      }
    }
  };
}

const monitorNodo1 = crearMonitorServidor(101);
monitorNodo1.registrarFallo(); // Nodo 101: Fallo #1
// monitorNodo1.contadorReintentos; // undefined (Totalmente encapsulado)
```

Garbage Collection y Memory Leaks en Closures: Si un closure hace referencia a un elemento del DOM que ha sido eliminado de la pantalla, o retiene un `setInterval` sin limpiar mediante `clearInterval()`, el colector de basura (*Mark-and-Sweep*) de V8 no podrá liberar esa memoria, provocando que la pestaña del navegador consuma gigabytes de RAM de forma progresiva.

4.1 MÓDULOS ES NATIVOS (ESM), IMPORTACIONES DINÁMICAS Y BUNDLING

ES MODULES ARCHITECTURE

CommonJS vs. ECMAScript Modules (ESM)

El Estándar Oficial de la Web: Históricamente, Node.js utilizaba `require()` y `module.exports` (CommonJS, síncrono). En el navegador y en los entornos modernos de desarrollo, el estándar universal es **ES Modules (ESM)** usando `import` y `export`. A diferencia de CommonJS, ESM es analizado estáticamente en la fase de parsing, lo que permite a empaquetadores como Vite o Rollup realizar *Tree-Shaking* (eliminación de código muerto que nunca se invoca en producción).

JAVASCRIPT: EXPORTACIÓN, IMPORTACIÓN Y DYNAMIC IMPORTS — modules-app.js

```
// --- ARCHIVO: utils/formatters.js (Exportaciones Nombradas) ---
export const VERSION_API = "2.4.0";

export function formatearBytes(bytes) {
  if (bytes === 0) return "0 B";
  const k = 1024;
  const tamanos = ["B", "KB", "MB", "GB"];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return `${parseFloat((bytes / Math.pow(k, i)).toFixed(2))} ${tamanos[i]}`;
}

// --- ARCHIVO: main.js (Consumo e Importación Dinámica Condicional) ---
import { formatearBytes, VERSION_API } from './utils/formatters.js';

console.log(`Inicializando cliente bajo ESM v${VERSION_API}`);

// Dynamic Import: Carga de código diferida bajo demanda (Code Splitting)
async function exportarReportePDF(datos) {
  try {
    // El módulo de PDF solo se descarga de la red cuando el usuario hace clic
    const moduloPDF = await import('./services/pdfGenerator.js');
    moduloPDF.generarDocumento(datos);
  } catch (error) {
    console.error("Fallo al cargar módulo de reportes:", error);
  }
}
```

Uso en HTML: Para que el navegador interprete archivos ESM nativos sin empaquetador, debes declarar obligatoriamente el atributo en la etiqueta del script: `<script type="module" src="js/main.js"></script>`. Los módulos ES se ejecutan automáticamente en modo diferido (`defer`) y con *strict mode* activo.

5.1 MANIPULACIÓN DEL DOM: DOCUMENTFRAGMENT Y PREVENCIÓN XSS

DOM REACTIVO

Layout Thrashing y Vulnerabilidades XSS

Por qué Evitar innerHTML sin Sanitizar: El uso ingenuo de `element.innerHTML = \`<h3>${userInput}</h3>\`` permite ataques de *Cross-Site Scripting (XSS)* si el dato proviene de un usuario o API comprometida (ej: inyectar `<img src=x onerror=alert(document.cookie)>`).
Layout Thrashing: Insertar elementos en un bucle directamente con `parent.appendChild()` obliga al motor a recalcular geometrías repetitivamente. Con `DocumentFragment` creamos un contenedor liviano fuera del DOM, ensamblamos todo en memoria y realizamos una única inserción en pantalla.

JAVASCRIPT: RENDERIZADOR SEGURO CON SANITIZACIÓN Y FRAGMENTOS — render-servidores.js

```
// Función auxiliar para neutralizar inyecciones de código HTML malicioso
export function escapeHtml(texto) {
  if (texto === null || texto === undefined) return "";
  const div = document.createElement('div');
  div.textContent = String(texto);
  return div.innerHTML;
}

// Renderizador de Tarjetas de Alto Rendimiento
export function renderizarServidores(servidores, contenedorId = "grid-nodos") {
  const contenedor = document.getElementById(contenedorId);
  if (!contenedor) return;

  // Limpiar contenido previo de forma rápida
  contenedor.textContent = "";

  if (servidores.length === 0) {
    const vacio = document.createElement('p');
    vacio.className = "alerta-vacia";
    vacio.textContent = "No se encontraron servidores coincidentes.";
    contenedor.appendChild(vacio);
    return;
  }

  // Contenedor fantasma en memoria (CERO REFLOWS HASTA EL FINAL)
  const fragmento = document.createDocumentFragment();

  servidores.forEach(srv => {
    const tarjeta = document.createElement('article');
    tarjeta.className = `node-card node-card--${srv.online ? "online" : "offline"}`;
    tarjeta.dataset.id = srv.id;

    tarjeta.innerHTML = `
      <header class="node-card__header">
        <h3 class="node-card__title">${escapeHtml(srv.hostname)}</h3>
        <span class="badge ${srv.online ? "badge--online" : "badge--offline"}">
          ${srv.online ? "ONLINE" : "OFFLINE"}
        </span>
      </header>
      <div class="node-card__body">
        <p>CPU: <strong>${Number(srv.cpu).toFixed(1)}%</strong></p>
      </div>
      <footer class="node-card__actions">
        <button class="btn btn--primary" data-action="restart">Reiniciar</button>
      </footer>
    `;
    fragmento.appendChild(tarjeta);
  });

  // UNA SOLA MUTACIÓN REAL EN EL DOM
  contenedor.appendChild(fragmento);
}
```

6.1 EL FLUJO DE EVENTOS: CAPTURA, TARGET Y EVENT DELEGATION

EVENT ARCHITECTURE

Fases del Event Flow: Capture vs. Bubbling

Propagación de Eventos en el DOM: Cuando el usuario hace clic en un botón, el evento viaja en 3 fases:
1. **Capture Phase (Captura):** El evento desciende desde `window` y `document` hacia abajo hasta el elemento objetivo.
2. **Target Phase:** El evento alcanza el nodo exacto que recibió el clic (`e.target`).
3. **Bubbling Phase (Burbujeo):** El evento asciende de regreso por todos sus ancestros hasta `window`.
Patrón Event Delegation: Aprovecha el burbujeo para registrar **un único listener en el contenedor padre**. Gestiona clics en cientos de elementos hijos presentes o que se añadan dinámicamente en el futuro.

JAVASCRIPT: DELEGACIÓN DE EVENTOS CON CLOSEST() Y DATA-ATTRIBUTES — event-delegation.js

```javascript
// Gestor Centralizado de Acciones sobre el Grid de Servidores
export function inicializarEventosGrid(contenedorId, despachadorAcciones) {
  const grid = document.getElementById(contenedorId);
  if (!grid) return;

  // UN SOLO EVENT LISTENER PARA TODAS LAS TARJETAS PRESENTES Y FUTURAS
  grid.addEventListener('click', (evento) => {
    // 1. Interceptar si el clic ocurrió dentro de un botón de acción
    const boton = evento.target.closest('button[data-action]');
    if (!boton) return;

    // 2. Encontrar la tarjeta padre para extraer su identificador
    const tarjetaPadre = boton.closest('article[data-id]');
    if (!tarjetaPadre) return;

    const servidorId = Number(tarjetaPadre.dataset.id);
    const accion = boton.dataset.action;

    // 3. Despachar la acción al controlador
    despachadorAcciones(accion, servidorId, boton);
  });
}

// Ejemplo de Uso Práctico:
inicializarEventosGrid("grid-nodos", (accion, id, boton) => {
  if (accion === "restart") {
    boton.disabled = true;
    boton.textContent = "Reiniciando...";
    console.log(`Enviando orden de reinicio para el nodo ID: ${id}`);
  }
});
```

Ventajas Críticas de la Delegación: Ahorra memoria al evitar cientos de llamadas a `addEventListener`, previene fugas de memoria cuando los nodos se destruyen dinámicamente y permite que nuevos servidores inyectados desde un WebSocket respondan a los clics inmediatamente sin re-vincular eventos.

7.1 TÉCNICAS DE RATE LIMITING EN EL CLIENTE: DEBOUNCE Y THROTTLE

PERFORMANCE OPS

Diferencia entre Debounce y Throttle

Control de Frecuencia de Ejecución:
• **Debounce (Anti-rebote):** Agrupa ráfagas de eventos consecutivos y solo ejecuta la función objetivo **después de que el usuario haya dejado de emitir eventos** durante un período determinado (ej: esperar 300 ms sin teclear antes de lanzar una búsqueda contra la API).
• **Throttle (Acelerador de Tasa):** Garantiza que una función se ejecute **como máximo una vez cada X milisegundos** durante un flujo continuo de eventos (ej: recalcular posiciones de un scroll o redimensionamiento de ventana a 60 FPS sin saturar la CPU).

JAVASCRIPT: IMPLEMENTACIONES PURAS DE DEBOUNCE Y THROTTLE — rate-limiters.js

```javascript
// Implementación Universal de Debounce
export function debounce(fn, esperaMs = 300) {
  let temporizador;
  return function (...args) {
    const contexto = this;
    clearTimeout(temporizador);
    temporizador = setTimeout(() => {
      fn.apply(contexto, args);
    }, esperaMs);
  };
}

// Implementación Universal de Throttle
export function throttle(fn, limiteMs = 200) {
  let enPausa = false;
  return function (...args) {
    if (!enPausa) {
      fn.apply(this, args);
      enPausa = true;
      setTimeout(() => { enPausa = false; }, limiteMs);
    }
  };
}

// Aplicación Práctica en Buscador de Servidores
const inputBuscador = document.getElementById("input-busqueda");

const ejecutarBusquedaServidor = debounce((evento) => {
  const termino = evento.target.value.trim();
  console.log(`Consultando telemetría para: "${termino}"`);
  // Aquí se dispara la petición fetch a la API
}, 300);

inputBuscador.addEventListener("input", ejecutarBusquedaServidor);
```

Ahorro de Carga en el Servidor: Sin debounce, si un operador escribe "servidor-principal" (18 caracteres), el cliente disparará 18 peticiones HTTP consecutivas en menos de 2 segundos. Con un debounce de 300 ms, se dispara **una sola petición final** con la palabra completa.

8.1 EL MOTOR V8 AL DESCUBIERTO: CALL STACK, MICROTASKS Y MACROTASKS

CONCURRENCY ENGINE

Mecánica del Bucle de Eventos (Event Loop)

¿Cómo maneja V8 la asincronía siendo Single-Threaded?
1. **Call Stack (Pila de Llamadas):** Ejecuta el código sincrónico frame por frame. Si una función está en la pila, nada más puede ejecutarse.
2. **Web APIs:** Temporizadores (`setTimeout`), llamadas de red (`fetch`) y eventos del DOM se despachan a hilos de fondo del navegador.
3. **Microtasks Queue (Prioridad Crítica):** Callbacks de Promesas (`Promise.then()`, `await`, `queueMicrotask`). **Regla Inviolable:** El Event Loop vacía COMPLETAMENTE todas las microtareas pendientes antes de procesar cualquier macrotarea o ciclo de renderizado.
4. **Macrotasks Queue (Task Queue):** Callbacks de `setTimeout`, `setInterval` y eventos de periféricos. Solo se despacha una por ciclo.

JAVASCRIPT: ANÁLISIS DEL ORDEN DE EJECUCIÓN DEL EVENT LOOP — event-loop-puzzle.js

```javascript
console.log("1. Sincrónico: Inicio en Call Stack");

setTimeout(() => {
  console.log("2. Macrotarea: Callback de setTimeout (Task Queue)");
}, 0);

Promise.resolve().then(() => {
  console.log("3. Microtarea: Promesa resuelta (Microtask Queue)");
}).then(() => {
  console.log("4. Microtarea Encadenada: Se vacía antes de cualquier timer");
});

queueMicrotask(() => {
  console.log("5. Microtarea Explícita: Misma prioridad alta");
});

console.log("6. Sincrónico: Fin del Script");

/* SALIDA EXACTA EN CONSOLA:
   1. Sincrónico: Inicio en Call Stack
   6. Sincrónico: Fin del Script
   3. Microtarea: Promesa resuelta
   5. Microtarea Explícita
   4. Microtarea Encadenada
   2. Macrotarea: Callback de setTimeout
*/
```

Pregunta de Arquitectura Onetel: ¿Por qué un bucle infinito en una microtarea (ej. `function loop() { Promise.resolve().then(loop); }`) congela la pestaña del navegador igual que un bucle `while(true)` sincrónico? Porque el Event Loop jamás cederá el control al motor de renderizado mientras existan microtareas pendientes en la cola.

9.1 PROMESAS, ASYNC/AWAIT Y CONTROL DE CANCELACIÓN CONCURRENTE

ASYNC & ABORT ENGINE

Async/Await como Azúcar Sintáctico y Manejo de Errores

De Callback Hell a Flujos Secuenciales: `async/await` convierte el código asíncrono en una estructura lineal legible similar al código síncrono. Cada `await` suspende la ejecución de la función asíncrona local y cede el hilo de ejecución al Event Loop hasta que la promesa se resuelve o rechaza. Los fallos se capturan limpiamente mediante bloques estándar `try...catch...finally`.

JAVASCRIPT: CLIENTE HTTP CON TIMEOUT REAL Y ABORTCONTROLLER — cliente-abortable.js

```javascript
// Cliente con soporte de cancelación y límite de tiempo estricto
export async function consultarConTimeout(url, { timeoutMs = 5000, signalExterno = null } = {}) {
  // Controlador nativo de cancelación de red de la Web API
  const controladorInterno = new AbortController();
  const temporizador = setTimeout(() => {
    controladorInterno.abort(new Error(`Timeout de red: Superado el límite de ${timeoutMs}ms`));
  }, timeoutMs);

  // Unificar señales si el usuario proporcionó su propia señal externa
  const signalFinal = signalExterno
    ? AbortSignal.any([controladorInterno.signal, signalExterno]) // ES2024
    : controladorInterno.signal;

  try {
    const respuesta = await fetch(url, { signal: signalFinal });

    // CRÍTICO: fetch NO rechaza la promesa ante códigos 404 o 500
    if (!respuesta.ok) {
      throw new Error(`Fallo HTTP ${respuesta.status}: ${respuesta.statusText}`);
    }

    return await respuesta.json();
  } catch (error) {
    if (error.name === "AbortError") {
      console.warn("Petición cancelada formalmente por el cliente.");
    }
    throw error;
  } finally {
    // Prevenir fugas de timers en memoria
    clearTimeout(temporizador);
  }
}
```

AbortSignal.any() (ECMAScript 2024): Permite cancelar una petición HTTP si se cumple cualquiera de dos condiciones independientes: que el temporizador de 5 segundos se agote O que el usuario presione el botón de "Cancelar Descarga" en la interfaz.

10.1 RESILIENCIA DE RED: BACKOFF EXPONENCIAL Y MANEJO DE ESTADOS

FETCH API ARCHITECTURE

La Regla de Oro de Fetch API y Fallos de Servidor

El Error Más Común con Fetch: Muchos desarrolladores asumen que un error `500 Internal Server Error` o un `404 Not Found` dispara automáticamente el bloque `catch`. **Esto es falso:** `fetch()` únicamente rechaza la promesa si ocurre un fallo catastrófico de red (cable desconectado, DNS caído o bloqueo de CORS). Si el servidor responde con error 500, la promesa se resuelve con `response.ok === false`. Es imperativo evaluar `response.ok` manualmente.

JAVASCRIPT: CLIENTE CON BACKOFF EXPONENCIAL AUTOMÁTICO — fetch-backoff.js

```javascript
// Cliente con Reintentos Exponenciales (Backoff: 1s, 2s, 4s...)
export async function fetchConReintentos(url, { maxIntentos = 3, esperaBaseMs = 1000 } = {}) {
  for (let intento = 1; intento <= maxIntentos; intento++) {
    try {
      const res = await fetch(url, { headers: { "Accept": "application/json" } });

      if (!res.ok) {
        // Errores 4xx (cliente) no se deben reintentar; errores 5xx (servidor) sí
        if (res.status >= 400 && res.status < 500) {
          throw new Error(`Error cliente HTTP ${res.status}: Consulta rechazada.`);
        }
        throw new Error(`Fallo transitorio servidor HTTP ${res.status}`);
      }

      return await res.json();
    } catch (err) {
      const esUltimoIntento = intento === maxIntentos;
      if (esUltimoIntento) {
        console.error(`[RED CRÍTICA]: Agotados los ${maxIntentos} intentos.`);
        throw err;
      }

      // Fórmula matemática de Backoff Exponencial con Jitter aleatorio
      const delay = esperaBaseMs * Math.pow(2, intento - 1) + Math.random() * 200;
      console.warn(`Intento ${intento} fallido. Reintentando en ${Math.round(delay)}ms...`);
      await new Promise(resolver => setTimeout(resolver, delay));
    }
  }
}
```

¿Qué es el Jitter? Es una pequeña variación aleatoria de tiempo que se suma al retardo del reintento. Evita el fenómeno de "Manada Estampida" (*Thundering Herd*), donde miles de clientes cuyos servidores cayeron se reconectan al milisegundo exacto al mismo tiempo, volviendo a tumbar la infraestructura.

LABORATORIO 1: MOTOR DE BÚSQUEDA REACTIVO EN CLIENTE CON DEBOUNCE

HANDS-ON LAB 1

Objetivo del Laboratorio: Implementar un motor de búsqueda en tiempo real que filtre servidores industriales por hostname y estado, utilizando operaciones funcionales inmutables y debouncing para no bloquear la UI.

JAVASCRIPT: BUSCADOR REACTIVO EN TIEMPO REAL (buscador-reactivo.js) buscador-reactivo.js

```
// Estado centralizado local
const estadoApp = {
  servidoresOriginales: [],
  filtroTexto: "",
  filtroEstado: "TODOS"
};

// Función pura de filtrado multicriterio
function aplicarFiltros(lista, texto, estado) {
  const termino = texto.toLowerCase();
  return lista.filter(srv => {
    const coincideTexto = srv.hostname.toLowerCase().includes(termino) || srv.ip.includes(termino);
    const coincideEstado = estado === "TODOS" || srv.estado === estado;
    return coincideTexto && coincideEstado;
  });
}

// Orquestador con Debounce para el Input
export function inicializarBuscador(datosServidores, renderCallback) {
  estadoApp.servidoresOriginales = [...datosServidores];

  const inputTexto = document.getElementById("input-filtro");
  const selectEstado = document.getElementById("select-estado");

  const actualizarVista = () => {
    const resultado = aplicarFiltros(
      estadoApp.servidoresOriginales,
      estadoApp.filtroTexto,
      estadoApp.filtroEstado
    );
    renderCallback(resultado);
  };

  // Debounce de 250ms en la entrada del teclado
  let timer;
  inputTexto.addEventListener("input", (e) => {
    clearTimeout(timer);
    timer = setTimeout(() => {
      estadoApp.filtroTexto = e.target.value;
      actualizarVista();
    }, 250);
  });

  selectEstado.addEventListener("change", (e) => {
    estadoApp.filtroEstado = e.target.value;
    actualizarVista();
  });

  // Render inicial
  actualizarVista();
}
```

Verificación de Rendimiento: Abre el monitor de rendimiento en DevTools. Al escribir a gran velocidad en el campo de texto, confirma que el filtrado solo se invoca una vez al detener el tecleo y que el Garbage Collector no sufre incrementos escalonados de memoria.

LABORATORIO 2: ARQUITECTURA DE SINGLE PAGE APPLICATION (SPA) DE TELEMETRÍA

HANDS-ON LAB 2

Objetivo del Laboratorio: Construir el núcleo de una Single Page Application (SPA) que consulte métricas de infraestructura cada 5 segundos, maneje indicadores de carga, gestione errores de red y permita cancelar peticiones activas.

JAVASCRIPT: ORQUESTADOR COMPLETO DE LA SPA (app-spa.js) — app-spa.js

```
// Estado reactivo central de la SPA
class TelemetryStore {
  #servidores = [];
  #cargando = false;
  #suscriptores = [];

  suscribir(callback) {
    this.#suscriptores.push(callback);
    callback(this.#obtenerEstado());
  }

  #notificar() {
    const estado = this.#obtenerEstado();
    this.#suscriptores.forEach(fn => fn(estado));
  }

  #obtenerEstado() {
    return { servidores: [...this.#servidores], cargando: this.#cargando };
  }

  async sincronizar(urlEndpoint) {
    this.#cargando = true;
    this.#notificar();

    try {
      const res = await fetch(urlEndpoint);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const datos = await res.json();
      this.#servidores = Array.isArray(datos) ? datos : (datos.data || []);
    } catch (err) {
      console.error("Fallo al sincronizar telemetría:", err.message);
    } finally {
      this.#cargando = false;
      this.#notificar();
    }
  }
}

// Inicialización en el DOM
document.addEventListener("DOMContentLoaded", () => {
  const store = new TelemetryStore();
  const spinner = document.getElementById("spinner-carga");
  const contenedor = document.getElementById("contenedor-grid");

  store.suscribir(({ servidores, cargando }) => {
    if (spinner) spinner.style.display = cargando ? "block" : "none";
    // Invocar renderizador de DocumentFragment aquí
  });

  // Polling regular cada 5 segundos
  store.sincronizar("http://127.0.0.1:8001/api/v1/servidores");
  setInterval(() => store.sincronizar("http://127.0.0.1:8001/api/v1/servidores"), 5000);
});
```

Patrón Observer / Store: Al aislar el estado en una clase `TelemetryStore` y suscribir la vista mediante callbacks reactivos, logramos desacoplar la capa de red del renderizado del DOM, permitiendo reutilizar el mismo motor en navegadores o aplicaciones Electron.

10.2 TESTING UNITARIO EN JAVASCRIPT MODERNO (NODE TEST RUNNER Y VITEST)

UNIT TESTING OPS

Testing de Funciones Puras sin Dependencias Pesadas

Node.js Native Test Runner (node:test): En Node.js contemporáneo ya no es obligatorio instalar librerías externas voluminosas para ejecutar pruebas unitarias. El módulo nativo `node:test` y `node:assert/strict` permite testear lógica funcional pura, promesas y lanzamientos de errores a muy alta velocidad.

JAVASCRIPT: SUITE DE PRUEBAS UNITARIAS NATIVAS — servidores.test.js

```
import { test, describe } from 'node:test';
import assert from 'node:assert/strict';
import { escapeHtml } from './render-servidores.js';

describe("Suite de Pruebas: Seguridad y Sanitización", () => {
  test("escapeHtml debe neutralizar tags <script> maliciosos", () => {
    const inputXSS = "<script>alert('hacked')</script>";
    const resultado = escapeHtml(inputXSS);
    assert.equal(resultado, "&lt;script&gt;alert('hacked')&lt;/script&gt;");
  });

  test("escapeHtml maneja valores nulos sin lanzar excepción", () => {
    assert.equal(escapeHtml(null), "");
    assert.equal(escapeHtml(undefined), "");
  });
});

describe("Suite de Pruebas: Filtrado Funcional", () => {
  const mockServidores = [
    { id: 1, hostname: "srv-core", estado: "ONLINE" },
    { id: 2, hostname: "srv-backup", estado: "OFFLINE" }
  ];

  test("Filtra correctamente por estado ONLINE", () => {
    const online = mockServidores.filter(s => s.estado === "ONLINE");
    assert.equal(online.length, 1);
    assert.equal(online[0].hostname, "srv-core");
  });
});
```

Ejecución en Terminal: Ejecuta directamente `node --test servidores.test.js`. No requiere Babel, ts-node ni dependencias de npm; se ejecuta de forma nativa instantáneamente.

10.3 PROTOCOLO DE AUDITORÍA: JAVASCRIPT DEFENSIVO EN PRODUCCIÓN

QUALITY AUDIT

Matriz de Validación Técnica de Código JavaScript

#	Criterio de Aceptación (Calidad de Código y Arquitectura)	Método de Verificación y Resultado Esperado
01	Inmutabilidad de Estado	Uso de `structuredClone` o spread; cero mutaciones destructivas en arrays.
02	Funciones Puras en Transformación	Métodos `.map()`, `.filter()` y `.reduce()` sin efectos secundarios globales.
03	Prevención de Memory Leaks	Todo `setInterval` y `addEventListener` no delegado posee rutina de destrucción.
04	Módulos ES Nativos (ESM)	Código desacoplado con `import/export`; scripts cargados con `type="module"`.
05	Inserción Eficiente con DocumentFragment	Cero bucles que agreguen elementos directamente al DOM (*Layout Thrashing*).
06	Sanitización Anti-XSS	Cero uso de `innerHTML` con variables crudas; sanitización con `escapeHtml`.
07	Event Delegation Activo	Un único listener en contenedor padre con `e.target.closest()`.
08	Debounce en Entradas de Texto	Buscadores con retardo mínimo de 250ms antes de consultar APIs o filtrar.
09	Comprensión del Event Loop	Microtareas de Promesas resueltas de forma ordenada sin bloquear el Call Stack.
10	Validación de response.ok en Fetch	Evaluación explícita de códigos HTTP 4xx y 5xx antes de llamar a `response.json()`.
11	Manejo de Cancelación (AbortController)	Peticiones prolongadas soportan interrupción mediante `signal` y timeout.
12	Testing Unitario Automatizado	Pruebas de funciones críticas pasando exitosamente con `node:test`.

Técnicas de Auditoría en Consola de DevTools

COMANDOS PARA DETECCIÓN DE RENDIMIENTO Y FUGAS DE MEMORIA — Chrome DevTools Console

```
# 1. Monitoreo de memoria heap activa en tiempo real:
console.log("Uso Heap:", (performance.memory?.usedJSHeapSize / 1048576).toFixed(2), "MB");

# 2. Medir tiempo de ejecución de algoritmos con precisión de microsegundos:
console.time("TiempoRenderizado");
renderizarServidores(datosPrueba);
console.timeEnd("TiempoRenderizado");

# 3. Monitorear eventos disparados en un nodo específico:
monitorEvents(document.getElementById("grid-nodos"), ["click"]);
```

El Estándar de Calidad Onetel: Cada componente dinámico debe superar el 100% de esta auditoría antes de fusionarse a la rama principal mediante Pull Request.

HOJA DE REFERENCIA RÁPIDA (CHEAT SHEET: JAVASCRIPT MODERNO)

REFERENCIA RÁPIDA

TEXT: SNIPPETS ESENCIALES PARA DESARROLLADORES SENIOR — JavaScript Modern Cheatsheet

```
# --- SINTAXIS Y TRANSFORMACIÓN INMUTABLE ---
const copia = structuredClone(objeto);           # Clonación profunda sin referencias
const filtrados = arr.filter(x => x.activo);     # Filtrado funcional
const transformados = arr.map(x => x.valor);     # Proyección de datos
const ordenados = arr.toSorted((a, b) => a - b); # Orden inmutable (ES2023)
const { prop = valorDefault, ...resto } = obj;   # Destructuring + Rest

# --- MANIPULACIÓN EFICIENTE DEL DOM ---
const frag = document.createDocumentFragment();  # Contenedor fantasma anti-reflow
parent.appendChild(frag);                        # Una sola inserción real
nodo.closest('selector');                        # Ascender al ancestro coincidente
nodo.matches('selector');                        # Evaluar si coincide con clase/id

# --- ASINCRONÍA Y RED ---
const ctrl = new AbortController();              # Controlador de cancelación
fetch(url, { signal: ctrl.signal });             # Vincular señal a la petición
ctrl.abort();                                    # Cancelar petición inmediatamente
await Promise.all([p1, p2]);                     # Concurrencia paralela estricta
await Promise.allSettled([p1, p2]);              # Concurrencia resiliente a fallos

# --- EVENT DELEGATION & TIMING ---
contenedor.addEventListener('click', (e) => {
  const btn = e.target.closest('button[data-id]');
  if (btn) ejecutarAccion(btn.dataset.id);
});
```

Matriz de Selección de Métodos Asíncronos

Método de Promesa	Comportamiento ante Errores	Escenario de Uso
`Promise.all()`	Falla rápido (*Fail-Fast*): si una sola falla, aborta todo de inmediato.	Cargas dependientes donde se requieren todos los datos obligatorios.
`Promise.allSettled()`	Espera a todas: retorna array de objetos con estado `fulfilled` o `rejected`.	Consultar telemetría de 10 servidores independientes donde algunos pueden estar caídos.
`Promise.race()`	Resuelve o rechaza con la primera promesa que termine.	Sistemas de timeout o carreras de espejos de datos.